

Mycelium-Inspired Mutualistic Adaptive Learning Systems (MMALS): A Full Research Chronicle from Bio-Inspired Functional Memory to Auditable Inferred-Context Continual Learning, Selector Reliability, and RC2O-v1 Regime-Safe Deployment

Guillaume Harbonnier

Draft research article and reproducibility package

Updated June 2026 – RC2O-v1 robust qualification

Abstract

This paper consolidates the research trajectory of Mycelium-Inspired Mutualistic Adaptive Learning Systems (MMALS), a bio-inspired continual-learning program that treats memory not as a static parameter snapshot but as a mediated functional exchange across specialists. The project began as a mycelium metaphor for routing and mutualistic exchange, then progressively became an experimental continual-learning harness with explicit baselines, functional drift diagnostics, inferred context, no-leakage policy selection, audit artifacts, and selector reliability analysis. The central thesis that emerged is that the relevant memory object is not merely the selected route, but the function computed through that route: routes, latent transformations, outputs, context posteriors, readout behavior, and deployment policies must be observed together. This version preserves the v0.1–v1.1-RC2L chronicle, the backbone-learning versus deployment-policy distinction, and the RC2M-alpha/FashionMNIST robust evidence, then updates the conclusion with the latest RC2O-v1 results. RC2O-v1 implements a feature-only regime estimator and a hard regime-conditional selector. Across the four robust MNIST-family stress tests available in this package narrative, RC2O-v1 passes the intended regime-dependent deployment tests: it releases proto/global no-repair behavior on clean or stable domain-shift regimes such as MNIST, RotatedMNIST, and PermutedMNIST, while it locks the guarded context-gap family on the ambiguous FashionMNIST regime. The strongest updated claim is therefore no longer only that the candidate pool is strong; it is that the first Phase-1 regime-safe controller has been robust-qualified on the MNIST-family four-dataset suite. The claim remains deliberately bounded: this is not production readiness, not open-world continual learning, and not a formal conformal guarantee. The next research branches are external-domain qualification, host-specialization auditing, continual regime replay, and RC2O-v2/RC3 conformal regime-conditional calibration.

Claim status

This is a research article and reproducibility-package draft, not a peer-reviewed paper and not a production certification. The strongest current claim is: *MMALS v1.1-RC2O-v1 is a robust-qualified Phase-1 regime-safe deployment controller on the MNIST-family four-dataset suite*. It separates feature-only regime estimation from hard deployment-policy selection. In clean or stable domain-shift regimes, it releases the simple proto/global no-repair policy; in ambiguous regimes, it blocks no-repair and locks the guarded context-gap family. This updates the earlier Option A conclusion: RC2JKb remains an important audit finding, RC2M-alpha remains a selector-reliability exploration, and RC2O-v1 is the current best controller candidate. The conformal calibration layer discussed in this paper is future work, not part of the current RC2O-v1 implementation.

Contents

1	Executive summary	5
2	Scientific motivation	5
3	Core architecture	6
3.1	Hosts, transformations, routes, and mediated latent state	6
3.2	Route drift, latent drift, and output drift	6
4	Backbone learning versus deployment-policy selection	7
4.1	Two phases	7
4.2	Formal separation	8
4.3	Engineering analogy	9
4.4	Why the distinction matters	9
4.5	Definition	10
4.6	Why this is a strong research concept	10
5	Human-understandable story: learning the platform versus choosing the runtime mode	10
5.1	The core idea in one paragraph	11
5.2	Phase A: learning capability	11
5.3	Phase B: choosing the runtime behavior	11
5.4	The car analogy for engineers	12
5.5	The software platform analogy	12
5.6	Formal statement	13
5.7	Why the distinction matters in the RC2 lineage	13
5.8	Why not choose everything during training?	13
5.9	The clearest definition	14
5.10	Story for a 12-year-old	14
5.11	Visual explanation	15
5.12	Why this is a strong research concept	15
6	Benchmark setting	15
7	Research chronicle: from v0.1 to v1.1-RC2O-v1	16
7.1	v0.1: structural routing toy model	16
7.2	v0.2: fairer baselines and capacity checks	16
7.3	v0.3: making the fungal medium real	16
7.4	v0.4: warm-up, delayed mutualism, and soft consolidation	17
7.5	v0.5: route memory and output distillation	17
7.6	v0.6: route memory under harder benchmarks	17
7.7	v0.7: functional route memory	17
7.8	v0.8: final evidence harness	18
7.9	v0.9: route-function disentanglement	18
7.10	v0.10: cost and scalability	18
7.11	v0.11: comparative evidence harness	19
7.12	v0.12: Hopfield and co-evolution controls	19
7.13	v0.13 and v0.13.1: co-evolution and warm weak CE	19

7.14	v0.14 family: inferred context transition	20
7.15	v1.0-RC1b: benchmark-ready inferred-context candidate	20
7.16	v1.1-RC2b: strict no-leakage policy selection	20
7.17	RC2c to RC2g: repairing the selector and boundary classes	21
7.18	RC2H: RC1b-anchored conservative selector	21
7.19	RC2I: dual-anchor conservative selector	22
7.20	RC2J to RC2JF: regime-aware selection and hard clarity	22
7.21	RC2JG and RC2JH: weak-boundary calibration and Energy Context Probe	22
7.22	RC2JK and RC2JKb: Pareto-safe selection and the limit of threshold patching	23
7.23	RC2L: selector-oracle gap and validation-feature reliability audit	24
7.24	RC2M-alpha and the limit of calibrated boundary-first selection	24
7.25	RC2N and the threshold ceiling	24
7.26	RC2O-v1: feature-only regime estimator and hard regime-conditional deployment	25
8	A unified interpretation	26
8.1	What is the memory object?	26
8.2	Why the selector became the bottleneck	27
9	Cost and engineering interpretation	27
10	Auditability and Hydro diagnostics	28
11	Failure analysis	28
11.1	RC2b failure: free strict selection over-selects fragile policies	29
11.2	RC2e failure: class 2 fixed, class 4 collapsed	29
11.3	RC2f failure: class 4 fixed, class 2 weak again	29
11.4	RC2g failure: tri-boundary balance improves smoke but not evidence	29
11.5	RC2H failure: conservative anchor helps but context-only overrides remain unsafe	29
11.6	RC2JK failure: Pareto fallback over-protects clean MNIST	29
11.7	RC2JKb failure: clean escape remains too permissive on FashionMNIST	30
11.8	Meta-failure: selector regret, not candidate-capacity collapse	30
12	Current best scientific statement	30
13	Next research roadmap	31
13.1	External-domain qualification	31
13.2	Axis A: host-specialization audit	31
13.3	Continual regime replay	31
13.4	RC2O-v2 / RC3: conformal regime-conditional calibrated selector	31
13.5	Cost and production-readiness gap	32
14	Response to the original four MMALS pillars	32
15	Limitations	33
16	Conclusion	33
17	Milestone table with child-level story	34
18	Reproducibility package contents	35

1 Executive summary

The story of MMALS is the story of a metaphor becoming a measurement program. The early metaphor was biological: a fungal medium connects several hosts, routes energy, exchanges nutrients, and preserves useful adaptations. The machine-learning translation became a continual-learning system with several specialist hosts, transformations, a routing matrix, a mediated latent state, and memory losses that preserve old behavior while learning new tasks.

The research program evolved through four major scientific transitions.

- T1. From route memory to functional memory.** Early versions asked whether preserving the route was enough. The answer became no: the same route can compute a different function if hosts or transformations drift.
- T2. From explicit task IDs to inferred context.** Early versions knew the task. The v1 direction asks whether the model can infer the situation from the input and memory.
- T3. From performance-only notebooks to evidence harnesses.** v0.11 and later introduced baselines, cost, parameters, runtime, forgetting, and reproducibility artifacts.
- T4. From free selection to conservative deployment.** RC2b–RC2JKb showed that selecting a policy only by validation proxy can choose fragile policies. The next selector must be anchored on safe guarded policies and allow overrides only with strong evidence.
- T5. From selector patching to selector reliability science.** The RC2J family made the selector more careful, but also revealed threshold ping-pong: a rule that helps FashionMNIST can regress MNIST, and a rule that protects MNIST can become too permissive on FashionMNIST. RC2L therefore changed the research question from “which hand-coded rule should win?” to “are the validation/audit features reliable enough to predict the candidate-pool oracle?”

Story for a 12-year-old

Imagine a forest where different trees are good at different things. A mushroom network connects them. At first we asked: “Did the mushroom use the same path as before?” Later we learned that this is not enough. The path can be the same, but the message carried along the path can change. So the real question became: “Did the forest still solve the old problem in the same useful way while learning a new problem?”

2 Scientific motivation

Continual learning studies how a model can learn a sequence of tasks without catastrophically forgetting earlier tasks. Classical approaches include regularization methods such as Elastic Weight Consolidation (Kirkpatrick et al., 2017), distillation-based methods such as Learning without Forgetting (Li and Hoiem, 2017), replay methods, progressive neural networks (Rusu et al., 2016), exemplar-based incremental classifiers (Rebuffi et al., 2017), and gradient memory methods (Lopez-Paz and Ranzato, 2017; Chaudhry et al., 2019). These methods are useful comparators, but they often frame memory as parameters, examples, regularization, or architectural isolation.

MMALS explores a different analogy: biological mycelial networks. Mycorrhizal networks can connect organisms and support exchange of resources or signals (Simard et al., 1997); fungal growth and transport networks have also inspired network optimization models (Bebber et al., 2007). The

analogy is not used as biological proof. It is used as an engineering prior: useful memory may emerge from mediated exchange across specialists rather than from one monolithic representation.

The architectural question is therefore:

Can a system of specialists, connected by a learned fungal medium, preserve useful functions while adapting to changing contexts?

The scientific question is stronger:

Can we measure, audit, and falsify that preservation in a no-leakage continual-learning benchmark?

Story for a 12-year-old

Suppose you have five friends. One is good at shoes, one at shirts, one at numbers, one at maps, and one at drawings. A mushroom messenger decides whose advice to combine. If the group learns a new game, we do not only check whether the messenger visited the same friend. We check whether the group still gives the right advice.

3 Core architecture

3.1 Hosts, transformations, routes, and mediated latent state

Let x be an input. MMALS uses N specialist hosts. Each host i produces a representation

$$g_i(x). \quad (1)$$

A fungal transformation T_i maps the host representation into a shared exchange space:

$$\tilde{z}_i = T_i(g_i(x)). \quad (2)$$

A learned router produces weights

$$r_i(x, c) = \frac{\exp(a_i(x, c)/\tau_r)}{\sum_j \exp(a_j(x, c)/\tau_r)}, \quad (3)$$

where c is a context signal. In early v0.x versions, c was effectively an explicit task identifier. In the v1 direction, c is inferred from x and memory. The mediated latent state is

$$z_f(x, c) = \sum_i r_i(x, c) T_i(g_i(x)). \quad (4)$$

A readout then predicts the class label from z_f .

3.2 Route drift, latent drift, and output drift

The v0.9 route-function disentanglement introduced three distinct measurements:

$$D_r = \mathbb{E}_{x,c} [\|r_{\text{current}}(x, c) - r_{\text{teacher}}(x, c)\|_2^2], \quad (5)$$

$$D_z = \mathbb{E}_{x,c} [\|z_{f,\text{current}}(x, c) - z_{f,\text{teacher}}(x, c)\|_2^2], \quad (6)$$

$$D_y = \mathbb{E}_{x,c} [D_{\text{KL}}(p_{\text{teacher}}(y|x, c) \| p_{\text{current}}(y|x, c))]. \quad (7)$$

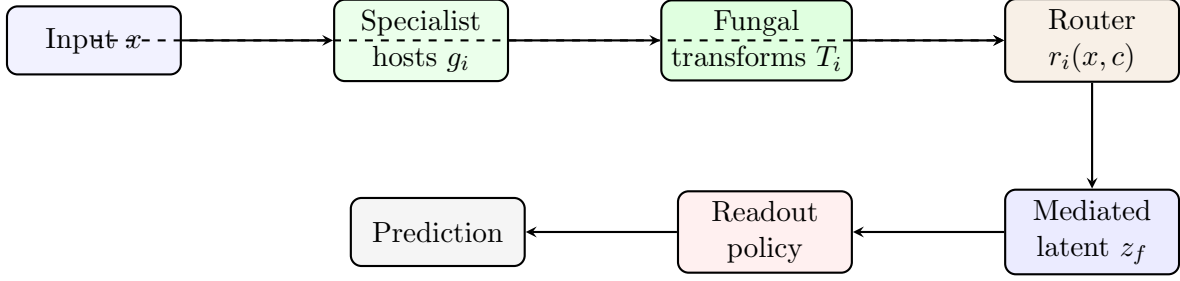


Figure 1: Conceptual MMALS flow. The route is only part of memory; the mediated latent function and readout behavior also matter.

A central non-implication is

$$r_{\text{current}}(x, c) \approx r_{\text{teacher}}(x, c) \not\Rightarrow z_{f, \text{current}}(x, c) \approx z_{f, \text{teacher}}(x, c). \quad (8)$$

Because the route can stay stable while the host or transformation changes, preserving route topology alone is insufficient.

Story for a 12-year-old

A bus can take the same road every day, but the passengers can be different. If the passengers are different, the trip has a different meaning. In MMALS, the route is the road, but the useful memory is the passengers plus the destination: what computation actually arrives.

4 Backbone learning versus deployment-policy selection

A central distinction in the current interpretation of MMALS is that training a continual-learning backbone is not the same operation as choosing the deployed policy. This distinction is both practical for engineering and theoretically important for the RC2 lineage.

4.1 Two phases

Phase A: backbone learning. During continual training, the model sees tasks sequentially:

$$\mathcal{T}_0 \rightarrow \mathcal{T}_1 \rightarrow \mathcal{T}_2 \rightarrow \mathcal{T}_3 \rightarrow \mathcal{T}_4. \quad (9)$$

In this phase, MMALS learns capability. It learns specialist host representations, fungal transformations, routing behavior, context posteriors, mediated latent states, memory prototypes, and readout parameters. Table 1 summarizes the learned objects.

At this stage, the question is:

Can the system represent old and new tasks in a useful way?

It is not yet the same as asking which final behavior should be trusted.

Phase B: deployment-policy selection. After the backbone has learned, the same trained system can be deployed through different candidate policies. Examples from the RC2 family include:

- `proto_global_no_repair`;

Table 1: Backbone learning: what is learned during sequential training.

Component	Role learned during continual training
Hosts $g_i(x)$	Specialized representations or transformations of the input.
Fungal transforms T_i	Maps host signals into a shared exchange space.
Router $r_i(x, c)$	Determines how much each host contributes under context c .
Context posterior $q_\theta(c x)$	Infers the situation from input and memory rather than from a task ID.
Mediated latent state z_f	Shared functional representation produced by routed host exchange.
Memory/prototypes	Compressed traces of old contexts, classes, and latent geometry.
Global readout/head	Maps mediated latent states to labels.

- `proto_global_head_ce_kl_guard_035`;
- `context_temp_blend_selected_head_guard.035`;
- `context_gap_selected`;
- `generic_weak_boundary_latent_calibrated`.

These are not necessarily independently trained full models. They are different ways of using, repairing, calibrating, guarding, or reading out the same learned backbone. The deployment-policy problem is therefore:

Given the trained MMALS system, which candidate behavior should be exposed as the final predictor?

4.2 Formal separation

Let the trained backbone be summarized by a mechanism

$$h_\theta(x) = \{g_i(x), T_i(g_i(x)), q_\theta(c | x), r_\theta(x, c), z_f(x, c)\}. \quad (10)$$

This mechanism contains what the model has learned: representations, context inference, routing, transformations, and mediated latent states. A deployment policy is a rule

$$\pi_k \in \Pi, \quad (11)$$

where Π is the candidate policy pool. Each π_k specifies a runtime configuration: which context posterior variant, which readout, which repair or calibration, and which guard should be used. The final predictor is therefore not merely

$$f_\theta(x), \quad (12)$$

but rather

$$f_{\theta, \pi_k}(x). \quad (13)$$

The no-leakage selector problem is to choose π_k using only training, repair, validation, and audit-memory information, before final-test evaluation.

4.3 Engineering analogy

For an engineer, backbone learning is like building the platform; deployment-policy selection is like choosing the runtime mode. The same trained platform can be activated in several modes.

Table 2: Engineering analogy: car platform versus driving mode.

Car world	MMALS world
Engine, sensors, controller	Backbone, hosts, transforms, router
Road condition	Inferred regime
Eco, sport, snow, off-road, safety mode	Candidate deployment policy
Dashboard and telemetry	Hydro, LSA, EER, Energy Probe, selector regret
Driving-mode selector	RC2 selector or future RC2L/RC2M meta-selector

If the road is dry, a fast mode may be appropriate. If the road is icy, the same car should use a safer mode. Similarly, if the inferred regime is clean, MMALS may safely release a simple proto/global policy. If the regime remains ambiguous, the selector should prefer a guarded context-gap policy.

In software terms, the trained backbone behaves like a reusable service with several runtime configurations:

```
--mode proto_no_repair
--mode guarded_global_head
--mode context_gap_guarded
--mode weak_boundary_calibrated
```

The engineering question is which mode should be activated for the current regime, using validation and audit signals rather than final-test results.

4.4 Why the distinction matters

A trained backbone can be good while the selected deployment policy is bad. This was a recurring finding in the RC2 lineage: the candidate pool often contained a strong behavior, but the selector sometimes exposed the wrong one. FashionMNIST-like ambiguous regimes favored the guarded context-gap family; MNIST-like clean regimes favored simpler proto/no-repair behavior. The failure was therefore not always a failure of learning. It was often a failure of selecting the correct runtime behavior.

This changes the diagnosis:

MMALS may have learned several usable behaviors, while still needing a more reliable selector to decide which one should be trusted.

Separating the phases is also useful because training and deployment selection optimize different objectives. During training, the system should learn flexible and reusable representations. During deployment selection, the system should choose safe, robust behavior under uncertainty. If these phases are mixed too early, several risks appear: over-regularization, over-repair of one weak class at the expense of another, validation overfit, hidden leakage, and loss of diagnostic clarity.

Table 3: Risks avoided by separating learning from deployment-policy selection.

Risk	Meaning
Over-regularization	The backbone becomes too rigid and loses useful plasticity.
Over-repair	A repair improves one weak boundary while damaging another.
Validation overfit	Candidate behavior adapts too tightly to a selector proxy.
Hidden leakage	Final evaluation becomes indirectly contaminated.
Lost diagnosis	It becomes unclear whether failure came from learning or policy selection.

4.5 Definition

Backbone learning is the process by which MMALS learns hosts, transformations, routes, context posteriors, mediated latent states, memory prototypes, and readout parameters across a sequence of tasks. Deployment-policy selection is the later no-leakage process that chooses, from a pool of candidate policies derived from the trained backbone, the behavior that will be evaluated and potentially deployed. The two must be separated because a continual-learning system can possess a good candidate behavior without reliably selecting it.

Story for a 12-year-old

Imagine a forest school. During the year, the mushroom network learns which trees are good at math, which trees are good at drawings, which paths connect trees, which old lessons should be remembered, and which situations look similar. That is training. Before the final exam, the mushroom must choose a strategy: a simple path, a guarded path, a careful-answer path, or a small bridge for a weak boundary. That is deployment policy. The mushroom can be smart and still choose the wrong strategy. So the problem is not always “the mushroom learned nothing.” Sometimes the problem is “the mushroom had several good strategies, but did not always know which one to use.”

4.6 Why this is a strong research concept

Many machine-learning papers talk as if the trained parameters are the final object. MMALS motivates a more system-oriented view: in continual learning, the final deployable object is the trained backbone plus the policy that decides how to use it under context uncertainty. This is the bridge from MMALS as a model to MMALS as an auditable adaptive learning system. It also clarifies why RC2L matters: RC2L does not primarily ask whether another model can be trained; it asks whether the system can predict which deployment policy should be trusted.

5 Human-understandable story: learning the platform versus choosing the runtime mode

This chapter restates Section 4 in a more human-readable form, because it is one of the most important distinctions in the RC2 story. Training a continual-learning backbone is not the same

thing as choosing the deployed policy. The distinction is PhD-level, but it is also very practical for engineers.

5.1 The core idea in one paragraph

MMALS training produces a reusable internal platform. That platform contains hosts, fungal transformations, routes, context posteriors, mediated latent states, memories, prototypes, and readouts. Deployment-policy selection then chooses the runtime configuration of that platform. The same trained backbone can be exposed through several candidate behaviors, such as `proto_global_no_repair`, `proto_global_head_ce_kl_guard_035`, `context_gap_selected`, or `generic_weak_bound`. Therefore the final scientific object is not only the trained parameters. It is the trained parameters plus the policy that decides how to use them under context uncertainty.

5.2 Phase A: learning capability

The first phase is ordinary continual backbone learning. The model sees a sequence of tasks,

$$\mathcal{T}_0 \rightarrow \mathcal{T}_1 \rightarrow \mathcal{T}_2 \rightarrow \mathcal{T}_3 \rightarrow \mathcal{T}_4, \quad (14)$$

and it learns whether it can represent old and new tasks in a useful way. In this phase, MMALS is not yet deciding which final policy should be trusted. It is learning the internal machinery that may later support several policies.

Table 4: Human-readable view of what the backbone learns.

Object learned	Meaning
Hosts $g_i(x)$	Specialist internal representations.
Fungal transforms T_i	How to map host signals into a shared exchange space.
Router $r_i(x, c)$	How much each host should contribute in a context.
Context posterior $q_\theta(c x)$	How the system infers the situation from input and memory.
Mediated latent state z_f	The shared functional representation that combines routed host signals.
Memory/prototypes	What old contexts, classes, and latent neighborhoods looked like.
Global readout/head	How to classify from the mediated representation.

In engineering language, this phase answers:

Can the system build enough reusable capability to solve old and new regimes?

5.3 Phase B: choosing the runtime behavior

After the backbone has learned, MMALS can be used in several ways. These ways are deployment policies. They are not necessarily different full models trained from scratch. They are different runtime configurations derived from the same trained backbone: a different context posterior, readout, repair, calibration, guard, or fallback decision.

The deployment-policy question is:

Given the trained MMALS system, which candidate behavior should be exposed as the final predictor?

This is why the selector became the bottleneck. A trained backbone can contain a good behavior, but the selector can still expose the wrong behavior.

5.4 The car analogy for engineers

Think of a car. Building the engine, sensors, braking system, and control system is capability learning. Choosing Eco mode, Sport mode, Snow mode, Off-road mode, or Safety mode is deployment-policy selection. The engine may be the same, but the runtime behavior is different.

Table 5: Car analogy for the backbone-versus-policy distinction.

Car world	MMALS world
Engine, sensors, controller	Backbone, hosts, transforms, router.
Road condition	Inferred regime.
Driving mode	Deployed policy.
Dashboard	Hydro, LSA, EER, Energy Probe, selector regret.
Driving-mode selector	RC2 selector or future RC2O-v2/RC3 calibrated selector.

If the road is dry, a faster mode may be acceptable. If the road is icy, Snow or Safety mode is better. If one always forces Snow mode, the car is too slow on dry roads. If one always forces Sport mode, it may be dangerous on ice. This is exactly the RC2 lesson: clean regimes should not be over-protected, while ambiguous regimes should not be under-protected.

5.5 The software platform analogy

For an engineer, the trained MMALS backbone behaves like a service with multiple runtime configurations:

```
--mode proto_no_repair
--mode guarded_global_head
--mode context_gap_guarded
--mode weak_boundary_calibrated
```

The engineering problem is not only training this service. It is choosing which runtime mode should be activated for the current data regime using validation and audit signals, not final-test results. This means that an auditable MMALS pipeline contains several layers:

1. a training pipeline for the backbone;
2. a candidate-generation pipeline for possible deployment policies;
3. a validation-scoring pipeline using repair and policy-validation memory;
4. a selector pipeline that chooses before final-test evaluation;
5. an audit/export pipeline that records why the decision was made.

This is why MMALS is becoming less like a single model and more like an auditable learning system.

5.6 Formal statement

The trained backbone can be summarized as

$$h_\theta(x) = \{g_i(x), T_i(g_i(x)), q_\theta(c | x), r_\theta(x, c), z_f(x, c)\}. \quad (15)$$

A deployment policy is an element

$$\pi_k \in \Pi, \quad (16)$$

where Π is the candidate policy pool. Each π_k specifies a context setting, readout/head, repair, calibration, guard, fallback, or selection behavior. The final predictor is therefore not simply $f_\theta(x)$, but

$$f_{\theta, \pi_k}(x). \quad (17)$$

The no-leakage selector problem is to choose π_k using validation and audit memory only, before final-test evaluation.

5.7 Why the distinction matters in the RC2 lineage

The RC2 lineage repeatedly showed that the candidate pool could contain the right behavior while the selector exposed the wrong behavior. This changed the diagnosis. The failure was not always that MMALS did not learn. It was often that MMALS learned several usable behaviors, but the selector chose the wrong deployed behavior.

Table 6: Why the selector became a separate research problem.

Dataset/regime	Strong candidate existed?	Selector problem observed in the lineage
FashionMNIST	Yes: guarded context-gap	Some selectors over-trusted simpler or true-guard behavior in ambiguous seeds.
MNIST	Yes: proto/simple	Some selectors over-locked to context-gap when no-repair should have been released.
PermutedMNIST	Yes: proto/simple	Mostly clean-domain behavior, but seed-level guards still needed audit.
RotatedMNIST	Yes: proto/simple and calibration candidate	Weak-boundary calibration remained useful as audited candidate evidence.

RC2O-v1 addresses this failure mode by explicitly separating feature-only regime estimation from hard regime-conditional deployment. It does not merely ask which candidate has the highest proxy score. It asks which regime is supported by the validation and audit traces, and then applies the deployment rule appropriate to that regime.

5.8 Why not choose everything during training?

Training and deployment selection optimize different things. During training, the system should learn flexible representations. During deployment selection, the system should choose safe and robust behavior under uncertainty. Mixing these phases too early can create the following risks.

Table 7: Risks of mixing backbone training and deployment-policy selection too early.

Risk	Meaning
Over-regularization	The model becomes too rigid and loses plasticity.
Over-repair	A repair fixes one weak class or boundary but damages another.
Validation overfit	Candidate behavior adapts too much to the selector proxy.
Hidden leakage	Final evaluation becomes indirectly contaminated.
Lost diagnosis	It becomes unclear whether failure came from learning or policy selection.

Separating the phases preserves the diagnosis:

Backbone training asks whether MMALS can learn useful functions. Policy selection asks which learned function should be trusted.

5.9 The clearest definition

Backbone learning is the process by which MMALS learns hosts, transformations, routes, context posteriors, mediated latent states, memory prototypes, and readout parameters across a sequence of tasks. Deployment-policy selection is the later no-leakage process that chooses, from a pool of candidate policies derived from the trained backbone, the behavior that will be evaluated and potentially deployed. The two must be separated because a continual-learning system can possess a good candidate behavior without reliably selecting it.

5.10 Story for a 12-year-old

Story for a 12-year-old

Imagine a forest school. The mushroom network is learning during the year. It learns which trees are good at math, which trees are good at drawings, which paths connect trees, which old lessons should be remembered, and which situations look similar. That is training. Before the final exam, the mushroom must choose a strategy: a simple path, a guarded path, a careful-answer path, or a small bridge path for a weak boundary. That is deployment policy. The mushroom can be smart and still choose the wrong strategy. If the forest is foggy and the mushroom has a safe guarded path, but it chooses the simple path because practice looked easy, then it may make mistakes on the final exam. So the problem is not always: “the mushroom learned nothing.” Sometimes the problem is: “the mushroom had several good strategies, but did not know which one to use.” That is why MMALS now studies the selector.

5.11 Visual explanation

Training phase	Policy-selection phase
Input examples	Backbone learned
↓	↓
Specialist trees / hosts	Candidate strategies
↓	↓
Mushroom routes	Practice-memory evaluation
↓	↓
Shared underground representation	Choose one strategy
↓	↓
Memory of old lessons	Final exam / final test
↓	
Backbone learned	

The candidate strategies are, for example, a simple path, a guarded global answer, a guarded context-gap path, or a small weak-boundary bridge. In MMALS notation, the same story is:

$$\text{sequential training} \rightarrow \theta = \{\text{hosts, transforms, router, context, memory, head}\}, \quad (18)$$

$$\theta \rightarrow \Pi = \{\pi_1, \pi_2, \pi_3, \pi_4\}, \quad (19)$$

$$\Pi \rightarrow \pi^* \rightarrow f_{\theta, \pi^*}. \quad (20)$$

The selector must choose π^* using validation memory, audit memory, and no final-test information.

5.12 Why this is a strong research concept

Many machine-learning papers talk as if the model after training is the final object. MMALS motivates a more system-oriented view: in continual learning, the final object is not only the trained parameters. It is the trained parameters plus the policy that decides how to use them under context uncertainty. This is a more mature engineering view and the bridge from MMALS as a model to MMALS as an auditable adaptive system. It also explains why the RC2L and RC2O lines matter: the question is no longer only whether another model can be trained; it is whether the system can predict which deployment policy should be trusted.

6 Benchmark setting

The current main stress benchmark is an overlap-chain class-incremental setting:

$$(0, 1) \rightarrow (1, 2) \rightarrow (2, 3) \rightarrow (3, 4) \rightarrow (4, 5). \quad (21)$$

Labels are global class IDs. This makes the problem harder than disjoint binary task-incremental learning because classes such as 2 and 4 sit at boundaries between contexts. Later notebooks support MNIST, FashionMNIST, RotatedMNIST, and PermutedMNIST.

Runs are staged:

- **smoke:** one seed, small samples, execution and artifact sanity.

- **evidence:** three seeds, larger samples, intermediate scientific signal.
- **robust:** five seeds, same larger samples, stronger qualification.
- **paper:** larger samples and seeds, intended only after the candidate is frozen.

Story for a 12-year-old

Instead of five separate school quizzes, the teacher makes a chain: quiz 1 has animals 0 and 1, quiz 2 has 1 and 2, quiz 3 has 2 and 3. Now class 2 appears in different neighborhoods. The model must understand the situation, not just memorize the quiz number.

7 Research chronicle: from v0.1 to v1.1-RC2O-v1

This section gives the full story in two layers: the PhD-level scientific role and the corresponding explanation for a 12-year-old.

7.1 v0.1: structural routing toy model

The first prototype established the basic idea: multiple hosts can be combined by a learned router, and the router can be interpreted as a mycelium-like medium. This was not yet evidence of continual learning. It was an architectural sketch.

Story for a 12-year-old

We built the first tiny forest. There were several trees and a mushroom path connecting them. The first question was simple: can the mushroom choose which tree to listen to?

7.2 v0.2: fairer baselines and capacity checks

The second step asked whether the apparent benefit was just capacity. The model needed to be compared against simpler neural baselines with comparable or at least explicit parameter counts. This began the discipline that later became central: never compare a mycelium-inspired model only against weak strawmen.

Story for a 12-year-old

If one team wins a race with a giant engine and the other team has a bicycle, the race is unfair. v0.2 started checking whether MMALS was winning because of the idea, or just because it had more parts.

7.3 v0.3: making the fungal medium real

The fungal medium became more than a metaphor. The architecture began to include specialists, transformations, routes, a mediated latent state, and mutualistic signals. The question became whether a learned medium can stabilize exchanges across hosts.

Story for a 12-year-old

The mushroom network stopped being just a drawing. It became a real messenger that could mix advice from different trees.

7.4 v0.4: warm-up, delayed mutualism, and soft consolidation

Early training can be fragile. v0.4 introduced warm-up and delayed mutualism so that the system would not over-constrain itself before useful host behavior emerged. The lesson was that mutualism must be timed.

Story for a 12-year-old

You should not ask a new team to make perfect teamwork on day one. First everyone learns their job; then the mushroom starts asking them to cooperate.

7.5 v0.5: route memory and output distillation

The next hypothesis was that if the router remembers old routing behavior and the outputs remain close to a teacher, old tasks should be preserved. Output distillation gave a behavioral anchor; route memory gave a mechanistic anchor.

Story for a 12-year-old

The mushroom tried to remember two things: which path it used before, and what answer the forest gave before. This is like remembering both the road and the final answer.

7.6 v0.6: route memory under harder benchmarks

v0.6 asked a causal question: if output distillation is already strong, does route memory add value on harder data? This was the start of a more skeptical style. A mechanism is not accepted because it sounds biologically plausible; it must improve or explain results.

Story for a 12-year-old

If remembering the answer already works, does remembering the path help? v0.6 checked whether the path memory was useful or just decorative.

7.7 v0.7: functional route memory

v0.7 moved from route preservation toward preserving the computation through routes. This is where the project began to converge on mediated functional memory: the system should preserve the old latent function, not merely the same route probabilities.

Story for a 12-year-old

We learned that it is not enough to say, “I used the same road.” We must ask, “Did the road still deliver the same package?”

7.8 v0.8: final evidence harness

v0.8 consolidated earlier variants into an evidence harness. It asked which variant performed best and exported results in a more reproducible form. This was the first move from exploratory notebooks to evidence packaging.

Story for a 12-year-old

Instead of trying one mushroom trick at a time, v0.8 organized a tournament and wrote down the scores.

7.9 v0.9: route-function disentanglement

v0.9 is a major conceptual step. It explicitly distinguishes route drift, functional latent drift, and output drift. It shows that route-only memory can fail badly even when more functional variants remain stable. In the v0.9 scorecard, the route-only control reached about 0.696 final accuracy with about 0.295 forgetting, while full functional memory reached about 0.991 final accuracy and near-zero forgetting.

Table 8: v0.9 route-function diagnostic summary.

Variant	Final accuracy	Forgetting
Full functional memory	0.9908	0.0004
Distillation only	0.9913	0.0004
Latent only	0.9905	0.0010
Route plus latent	0.9903	0.0011
Route memory only	0.6959	0.2951

The conclusion is not that route information is useless. It is that route information alone is not the memory object. Functional memory must be defined at the level of mediated latent computation and behavior.

Story for a 12-year-old

The mushroom remembered the map, but the forest still gave wrong answers. That proved the map alone is not memory. The answer depends on what travels through the map.

7.10 v0.10: cost and scalability

v0.10 made the cost question explicit. The model was larger and slower than simple MLP baselines. The scientific trade-off became clear: MMALS must justify its complexity through robustness, auditability, context handling, and functional memory, not just raw accuracy.

Story for a 12-year-old

A big forest is expensive to maintain. If it is only a little better than a small garden, maybe it is not worth it. v0.10 asked: what do we get for the extra trees?

7.11 v0.11: comparative evidence harness

v0.11 compared MMALS against known continual-learning baselines. In a smoke configuration, full functional memory reached 0.967 final accuracy and 0.003 forgetting, behind joint upper bound, PNN, and experience replay, but ahead of LwF, EWC, fine-tuning, and sparse MoE in that run. It used about 1.23M parameters versus about 218k for the MLP baselines.

Table 9: v0.11 comparative smoke result, selected rows.

Method	Final acc.	Forgetting	Params	Train time (s)
Joint upper bound	0.980	0.000	218k	1.02
PNN	0.979	0.000	218k	0.37
Experience replay	0.971	0.002	218k	0.49
MMALS full functional memory	0.967	0.003	1.23M	3.99
LwF	0.962	0.003	218k	0.65
EWC	0.873	0.102	218k	0.46
Fine-tuning	0.821	0.142	218k	0.41
Sparse MoE	0.795	0.176	1.19M	1.72

The key lesson was honest: MMALS was not automatically the best on simple benchmarks. Its value proposition must be harder settings, inferred context, auditability, and functional memory.

Story for a 12-year-old

The mushroom forest did well, but it did not beat every simpler team. That is okay. It told us the mushroom idea must prove itself on harder races, not just easy ones.

7.12 v0.12: Hopfield and co-evolution controls

The Hopfield branch tested whether memory should be understood as a frozen landscape. The evidence suggested that hard frozen-landscape memory is not the main answer. Hopfield-style ideas remain useful as diagnostic controls, but the deployable path remains functional mediated adaptation rather than simply freezing transformations.

Story for a 12-year-old

We tried turning part of the forest into stone so it would never change. It remembered some things, but it was not the best way to keep learning. A living forest needs some stable paths and some flexible growth.

7.13 v0.13 and v0.13.1: co-evolution and warm weak CE

The co-evolution branch explored transform drift, Jacobian drift, and resistance-like quantities inspired by geometry and physical systems. Warm weak CE ablations showed diagnostic value but did not yet establish a superior training recipe. The research lesson was that geometry can be a microscope before it becomes an optimizer.

Story for a 12-year-old

We gave the mushroom a microscope to watch how the paths bend and stretch. The microscope helped us see problems, but it was not yet the best tool to fix them.

7.14 v0.14 family: inferred context transition

The major conceptual shift before v1.0 was context inference. Earlier versions were told the task. The v1 direction asks the model to infer the situation from the input and memory, for example through a posterior $q(c|x)$. A small group of examples can be compared with old memory prototypes to infer context.

Story for a 12-year-old

Before, someone told the mushroom, “This is math class.” Now the mushroom must look around: rulers, triangles, numbers on the board. It has to guess: “This is probably math class.”

7.15 v1.0-RC1b: benchmark-ready inferred-context candidate

v1.0-RC1b consolidated a proto-first MMALS backbone, validation-selected context temperature/blend, and a guarded global head. It made the jump from explicit task IDs toward inferred-context class-incremental stress. The best candidate family became the guarded context-gap policy, often represented as `paired_context_gap_selected` or `context_temp_blend_selected_head_guard_035`.

Story for a 12-year-old

The mushroom no longer gets a label saying which classroom it entered. It looks at the objects, compares them with old memories, and chooses the path that seems most likely.

7.16 v1.1-RC2b: strict no-leakage policy selection

RC2b introduced a strict policy-selection protocol. It trains the same backbone, splits memory into repair, context-selection, and policy-validation partitions, builds candidate policies, scores them on validation memory only, selects before final-test evaluation, and exports audit traces. This is a major methodological improvement.

In the RC2b FashionMNIST robust run, the fixed RC1b context-gap candidate reached 0.9010 final average accuracy, 0.8183 min-task accuracy, 0.0556 forgetting, and 0.8235 class-2 accuracy. The RC2b automatic selected policy reached 0.8880 final average accuracy and still beat replay, but failed to match fixed RC1b.

Table 10: RC2b robust FashionMNIST selected results.

Method	Final acc.	Min task	Forgetting	Class 2	Class 5
Fixed RC1b context-gap	0.9010	0.8183	0.0556	0.8235	0.9985
RC2b selected policy	0.8880	0.7773	0.0644	0.7848	0.9985
Experience replay	0.8390	0.6985	0.1352	0.6188	0.9970

Story for a 12-year-old

The mushroom made a rule: before looking at the final exam, it chooses its strategy using only practice memories. That is fair. But sometimes it chose the wrong strategy, so the rule was fair but not yet smart enough.

7.17 RC2c to RC2g: repairing the selector and boundary classes

RC2c through RC2g were debugging branches:

- RC2c added risk-aware penalties, class-2 and min-task floors, no-repair vetoes, and hysteresis.
- RC2d aligned the selector to gates and fixed Drive archiving.
- RC2e made the selector NaN-safe and improved class-2 behavior.
- RC2f protected class 4 after class-4 collapse.
- RC2g introduced tri-boundary equilibrium across classes 2, 4, and 5.

RC2g smoke looked promising, but RC2g evidence showed the danger of free validation selection. The fixed RC1b context-gap candidate reached about 0.9058 final accuracy in evidence mode, while RC2g selected about 0.8704. The selector, not the backbone, was the issue.

Story for a 12-year-old

The mushroom kept trying new voting rules. One rule protected class 2 but hurt class 4. Another protected class 4 but hurt class 2. RC2g tried to balance the three difficult children: 2, 4, and 5. It helped, but the mushroom still sometimes picked a worse rule than the old safe one.

7.18 RC2H: RC1b-anchored conservative selector

RC2H inverted the selection question. Instead of asking which candidate has the highest proxy score, it asks whether there is enough validation-memory evidence to override the known-good RC1b anchor. It uses the guarded context-gap family as a default deployable anchor and permits overrides only when safety gates are satisfied.

In FashionMNIST evidence mode, RC2H validated the infrastructure but did not yet succeed as a performance selector. The true global guard was best in that run, the RC1b context-gap anchor was second, and RC2H selected policy remained replay-beating but weaker than the best deployable candidate.

Table 11: RC2H evidence FashionMNIST summary.

Method	Final acc.	Min task	Forgetting	Class 2	Class 5
True global guard	0.8957	0.8058	0.0623	0.8863	0.9958
RC1b context-gap anchor	0.8858	0.7908	0.0772	0.8429	0.9958
RC2H selected policy	0.8788	0.7358	0.0958	0.7696	0.9942
Experience replay	0.8361	0.6838	0.1415	0.6054	0.9975

The lesson is that the anchor should likely become a dual safe-anchor family: context-gap plus true global guard. Context-only global candidates should remain audit candidates, not deployment overrides, unless much stronger evidence exists.

Story for a 12-year-old

The mushroom learned a safer rule: “Do not change from the trusted plan unless the new plan is clearly better.” But two times it still trusted a shortcut that looked good in practice and was weaker on the final test. Next, the mushroom should trust two safe teachers instead of one.

7.19 RC2I: dual-anchor conservative selector

RC2I generalized the RC2H intuition into a dual-anchor selector. The key idea was that a single safe anchor is sometimes too narrow: FashionMNIST had strong evidence for the guarded context-gap family, while other regimes could prefer a guarded global CE/KL head or a simpler proto policy. RC2I therefore preserved the no-leakage selection discipline while adding explicit dual-anchor audit columns and seed-level safety traces.

Story for a 12-year-old

The mushroom learned that one trusted teacher is not always enough. Sometimes the safe teacher is the fog-guarded path; sometimes it is the simple guarded path. RC2I started keeping both teachers on the table.

7.20 RC2J to RC2JF: regime-aware selection and hard clarity

RC2J made the regime question explicit. The selector was no longer only asking which candidate has the largest proxy score; it was asking whether the current checkpoint looks like a clean regime, an ambiguous/foggy regime, or a mixed regime. RC2Jc introduced the final-checkpoint ambiguity lock for FashionMNIST-like cases. RC2Jd added the clean-context veto after MNIST showed that a lock can be over-protective. RC2JE hardened that veto: the selector may call the forest clear only when multiple validation-memory signs agree. RC2JF then added a safe true-guard veto for PermutedMNIST-like cases where the true global CE/KL guarded head is validated as safer than context-gap.

The scientific lesson is important: no-leakage is necessary but not sufficient. A selector can be fair and still wrong. It must also be regime-aware, weak-class-aware, and conservative under ambiguity.

Story for a 12-year-old

Sometimes the forest is foggy, so the mushroom should keep the guarded path. Sometimes the forest is clear, so the mushroom should not over-protect. RC2J through RC2JF were attempts to teach the mushroom the difference.

7.21 RC2JG and RC2JH: weak-boundary calibration and Energy Context Probe

RC2JG added a generic weak-boundary latent separability calibration candidate. It was not a RotatedMNIST-specific or class-5-specific repair. The trigger, training, and selection remain

validation-memory-only: detect a weak boundary in memory, train a small calibration candidate on repair memory, and allow it to win only if no-regression gates pass on policy-validation memory.

RC2JH added the Energy Context Probe as an audit-only diagnostic. The probe freezes the observed latent state and memory state, optimizes only a continuous context vector, and asks whether the energy landscape contains a recoverable basin near the oracle context. The important outcome of the MNIST/FashionMNIST evidence was not that the probe should become a training loss. It was that the landscape is often well formed, while raw context-space distances can remain scale-sensitive. This motivated context-space scale audits, normalized gap, cosine-to-oracle, norm ratio, and energy drop ratio.

Story for a 12-year-old

Sometimes the mushroom chooses the right path, but two roots are still too close together. RC2JG tries a small bridge, but only using practice memories. RC2JH adds a compass: if the mushroom searches for the right context, does the terrain guide it toward a good valley?

7.22 RC2JK and RC2JKb: Pareto-safe selection and the limit of threshold patching

RC2JK introduced a Pareto-safe context-gap fallback: in ambiguous validation regimes, guarded context-gap becomes the safety floor and challengers may override it only if they are not worse on final validation proxy, min-task proxy, weak-class behavior, Hydro turbulence/output drift, and LSA. This was mechanically correct but too conservative on MNIST, where it over-locked to the context-gap family.

RC2JKb added a clean-regime escape and strict ambiguity floor. It separated context uncertainty from performance-risk ambiguity. This was a real audit improvement: context uncertainty alone no longer forced guarded fallback. However, FashionMNIST robust evidence showed that RC2JKb remained too permissive in seeds where clean escape selected true-guard while context-gap guarded was the best deployable candidate. Thus RC2JKb is not the final selector. It is an audit finding: it proves the distinction between context uncertainty and performance-risk ambiguity, but it does not yet solve universal policy selection.

Table 12: Conservative multi-dataset conclusion after RC2JKb. Values are robust five-seed summaries from the candidate-pool evidence used for the Option A publication path.

Dataset/regime	Best deployable policy	Scientific status	Residual risk
FashionMNIST	guarded context-gap	strongest ambiguous-regime candidate	selector may over-trust true-guard
MNIST	proto/no-repair or simple guarded	clean-regime candidate	fallback can over-lock
PermutedMNIST	proto/simple global	clean domain-shift candidate	monitor seed-level class 2
RotatedMNIST	proto/no-repair; calibration candidate	geometric-shift candidate	latent boundary remains an audit issue

Story for a 12-year-old

The mushroom became better at telling fog from risk, but it still sometimes trusted the wrong shortcut. That does not mean the forest failed. It means the voting rule is now the weak part.

7.23 RC2L: selector-oracle gap and validation-feature reliability audit

RC2L changes the research program. Instead of adding another hand-coded selector patch, it builds an offline selector dataset from robust evidence packages. For each dataset, seed, checkpoint, and candidate, it extracts validation-memory and audit-memory features: proxy accuracy, min-task proxy, weak-class proxy, boundary EER/AUC, Hydro turbulence and output drift, LSA alignment, Energy Context Probe metrics, and RC2JKb margins. It then compares the selected candidate with the candidate-pool oracle and measures selector regret.

This is an audit-only procedure. It does not feed final-test information back into training or deployment. Its purpose is to answer a falsifiable question: are the current validation/audit features reliable enough to predict the best deployable candidate across datasets? If leave-one-dataset-out selector regret is low, RC2M can become a calibrated meta-selector. If regret remains high, the problem is not another threshold; the problem is the evidence layer itself.

Story for a 12-year-old

Instead of inventing one more mushroom rule, RC2L asks: from all the practice notes, can we tell which rule would have worked best? If yes, the mushroom can learn a better rule. If no, it needs better practice notes.

7.24 RC2M-alpha and the limit of calibrated boundary-first selection

RC2M-alpha tested whether the selector-reliability evidence could be turned into a calibrated boundary-first selector. It preserved the no-leakage discipline and exported candidate-feature datasets, weight-grid summaries, leave-one-dataset-out selector summaries, and a robust Fashion-MNIST package. This branch was important because it moved the selector away from ad hoc single-threshold repairs and toward a candidate-feature view of deployment policy selection.

However, RC2M-alpha remained an intermediate step. It improved the experimental infrastructure and showed how validation/audit features can be assembled into a selector dataset, but it did not yet resolve the core conflict between clean-regime release and ambiguous-regime protection. It still belonged to the family of selector-scoring approaches: useful for audit, but not yet the final regime-safe controller.

Story for a 12-year-old

RC2M-alpha was like giving the mushroom a spreadsheet of practice clues. That was better than guessing from one clue, but it still did not fully teach the mushroom when the forest was truly clear and when it was still foggy.

7.25 RC2N and the threshold ceiling

RC2N explored increasingly explicit clean-context release gates. The scientific value of RC2N was that it produced a controlled failure pair. One side of the problem was a false lock: in a clean MNIST seed, the selector blocked no-repair even though no-repair was the correct deployable behavior. The

other side was a false release: in FashionMNIST seed 2, the selector released no-repair when the guarded context-gap family was safer.

This pair is important because it shows why another global threshold is not enough. A stricter threshold protects FashionMNIST but over-locks MNIST. A looser threshold releases MNIST but can under-protect FashionMNIST. The solution cannot simply be “more conservative” or “more aggressive.” The solution must be regime-conditional.

Story for a 12-year-old

The mushroom learned that one rule cannot fit all weather. If it is too careful, it crawls on a sunny day. If it is too brave, it slips in the fog. It needs to first understand the weather, then choose the path.

7.26 RC2O-v1: feature-only regime estimator and hard regime-conditional deployment

RC2O-v1 is the current best controller in this chronicle. It implements the missing separation between two questions:

1. **Regime estimation:** what kind of situation does the validation/audit memory indicate?
2. **Deployment choice:** given that regime, which policy family is safe to expose?

The regime estimator is feature-only: it is not allowed to use the dataset name, the final-test result, or a post-hoc oracle ranking. It uses validation and audit traces such as context entropy, context confidence, proxy min-task behavior, proxy dispersion, Hydro turbulence, Hydro output drift, Energy Probe status when available, and other memory-only safety signals. The hard selector then applies a deliberately simple deployment rule:

- if the regime is `CLEAN_CONTEXT`, release `proto_global_no_repair` when safety checks pass;
- if the regime is `AMBIGUOUS_CONTEXT`, block no-repair and lock the guarded context-gap family;
- if the regime is `UNKNOWN_OR_MIXED`, abstain to the safe guarded context-gap family.

This is not a conformal selector yet. It is a robust-qualified rule-based safe selector. The conformal layer discussed later would transform the same audit signals into calibrated nonconformity scores. RC2O-v1 instead uses explicit hard regime-conditional rules.

Table 13: RC2O-v1 robust qualification across the four MNIST-family regimes. The MNIST row is included from the completed robust summary; the other rows are extracted from the mounted robust packages.

Dataset	Inferred regime		Final acc.	Min task	Forgetting	Class 2	Class 5	Action
MNIST	CLEAN		0.9646	0.9500	0.0198	0.9528	0.9325	release
FashionMNIST	AMBIGUOUS	/	0.9010	0.8183	0.0537	0.8235	0.9985	guarded lock
	MIXED							
RotatedMNIST	CLEAN		0.9678	0.9480	0.0215	0.9418	0.9585	release
PermutedMNIST	CLEAN		0.9698	0.9405	0.0221	0.9478	0.9655	release

The key achievement is not merely the average accuracy. It is the fact that the same selector performs two opposite actions correctly. On MNIST, RotatedMNIST, and PermutedMNIST, it

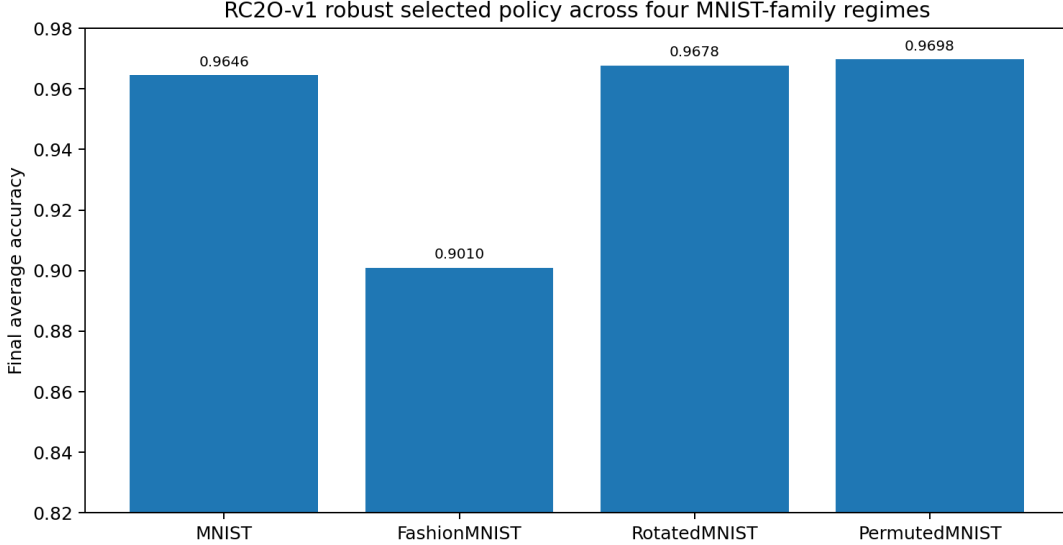


Figure 2: RC2O-v1 selected-policy final average accuracy across the four robust regimes.

avoids over-protection and releases the simple global/no-repair policy. On FashionMNIST, it blocks the dangerous no-repair release and locks the guarded context-gap family. This directly addresses the RC2N failure pair.

Table 14: RC2O-v1 selector-regret interpretation. Regret is measured against the best deployable candidate in the candidate pool.

Dataset	RC2O final acc.	Regret	Interpretation
MNIST	0.9646	0.0003	fixes the old clean-regime false lock, with negligible residual regret
FashionMNIST	0.9010	0.0048	blocks the dangerous false release; residual regret is mainly conservative seed-1 fallback
RotatedMNIST	0.9678	0.0001	nearly exact best deployable selection under geometric shift
PermutedMNIST	0.9698	0.0005	clean domain-shift release, essentially tied with the best deployable family

Story for a 12-year-old

RC2O-v1 is the mushroom learning the weather rule. When the forest is clear, it does not force the slow guarded path. When the forest is foggy, it refuses the dangerous shortcut. The important result is that it can do both with the same rule system.

8 A unified interpretation

8.1 What is the memory object?

The main scientific result of the research program is not a single accuracy number. It is the clarification of the memory object. MMALS memory has at least five layers:

1. **Route memory:** which hosts are weighted.
2. **Transformation memory:** how host signals are mapped into exchange space.
3. **Latent functional memory:** what mediated representation z_f is computed.
4. **Output behavioral memory:** whether predictions remain stable.
5. **Context-posterior memory:** whether the system infers the right situation.

A deployment policy adds a sixth layer: the readout/head and its calibration. The RC2 lineage showed that even a strong backbone can be undermined by a weak selector or readout policy.

Story for a 12-year-old

Memory is not just “which road did I take?” It is the road, the package, the messenger, the final answer, and whether the mushroom understood which village it was visiting.

8.2 Why the selector became the bottleneck

RC2b through RC2JKb show a repeated pattern. The backbone is often capable; diagnostic probes reach high accuracy; fixed guarded policies perform well. But validation-memory policy selection can overfit proxy signals. A policy can look good on validation memory but damage weak boundary classes in final evaluation. The updated conclusion is sharper: the candidate pool is often strong enough, while the selector has non-zero regret relative to the candidate-pool oracle.

This creates a deployment principle:

No-leakage selection is necessary but not sufficient. A selector must also be conservative, anchor-aware, weak-class aware, and reliability-calibrated. RC2L therefore treats the selector itself as an object of measurement.

Story for a 12-year-old

Not cheating on the exam is necessary. But even if you do not cheat, you can still choose the wrong study strategy. RC2H is about choosing a strategy carefully, not just fairly.

9 Cost and engineering interpretation

MMALS is not free. The v0.11 and RC2 runs make parameter cost visible: MLP baselines are around 218k parameters, while MMALS variants are around 1.2M to 1.34M parameters. The deployment question is therefore not “does MMALS beat everything on every toy benchmark?” It is:

Does MMALS provide enough robustness, context inference, auditability, and functional-memory control to justify its higher cost in harder continual-learning settings?

Story for a 12-year-old

A big forest costs more than a small garden. So the big forest must do something the garden cannot: explain itself, survive harder weather, and remember old paths while growing new ones.

Table 15: Cost-oriented comparison from selected evidence packages.

Method family	Example final acc.	Params	Model size
MLP replay baseline	0.836–0.971	~218k	~0.83 MB
PNN baseline	0.979 in v0.11 smoke	~218k	~0.83 MB
Sparse MoE	0.795 in v0.11 smoke	~1.19M	~4.54 MB
MMALS functional memory	0.967 in v0.11 smoke	~1.23M	~4.70 MB
MMALS RC2 family	0.879–0.901 in harder overlap-chain	~1.34M	~5.10 MB

Table 16: Current robust evidence after RC2O-v1. RC2O-v1 is the selected deployment controller; it is no longer only an audit finding.

Dataset / selected policy	Final acc.	Min task	Forgetting	Class 2	Class 5
MNIST RC2O-v1 selected	0.9646	0.9500	0.0198	0.9528	0.9325
FashionMNIST RC2O-v1 selected	0.9010	0.8183	0.0537	0.8235	0.9985
RotatedMNIST RC2O-v1 selected	0.9678	0.9480	0.0215	0.9418	0.9585
PermutedMNIST RC2O-v1 selected	0.9698	0.9405	0.0221	0.9478	0.9655

10 Auditability and Hydro diagnostics

The v1.1 Hydro branch adds a fluid-inspired diagnostic layer: route drift, latent drift, output drift, adaptive viscosity, Reynolds-like instability, turbulence-like scores, and wave/context entropy. The current conclusion is conservative. Hydro is useful as an audit layer, not yet as the default performance policy.

Hydro metrics matter because they create observability. A model that learns continually should not only output an accuracy number; it should explain whether routes, latent states, outputs, and context posterior were stable or turbulent.

Story for a 12-year-old

A river can look calm or turbulent. Hydro gives the mushroom forest a water-meter. It tells us whether the learning flow is smooth or chaotic. For now it is a dashboard, not the steering wheel.

The later RC2J line adds two more audit layers. **Learning-Signal Alignment** (LSA Panel E) compares expected update allocation across hosts against observed gradient/update norm allocation. It asks whether the model is learning in the modules that the route, gain, and memory-risk signals suggest should adapt. **Energy Context Probe** freezes a seed/checkpoint state and probes whether a recoverable context basin exists. In robust runs, these diagnostics generally did not indicate global learning-signal collapse or energy-landscape failure; the remaining bottleneck was selection reliability.

11 Failure analysis

The recent selector lineage produced valuable failures.

Table 17: Audit-layer interpretation after RC2JH/RC2JKb.

Audit layer	What it observes	Current interpretation
Hydro	route/latent/output drift, turbulence	observability, not default
LSA Panel E	expected vs observed update allocation	no global learning-signal
EER diagnostics	one-vs-rest weak-boundary reliability	class 2 remains a key boundary
Energy Context Probe	context-energy basin and scale	landscape often healthy, stable
RC2L selector audit	selector regret vs candidate oracle	reliability audit branch
RC2O-v1 selector	feature-only regime label, hard release/lock/fallback action	current Phase-1 controller

11.1 RC2b failure: free strict selection over-selects fragile policies

RC2b was methodologically correct but not performance-optimal. It selected one policy per checkpoint without final-test leakage, but selected weaker policies too often.

11.2 RC2e failure: class 2 fixed, class 4 collapsed

Class-specific rescue can overcorrect. Protecting one boundary class can sacrifice another.

11.3 RC2f failure: class 4 fixed, class 2 weak again

Boundary protection must be symmetric and equilibrium-oriented.

11.4 RC2g failure: tri-boundary balance improves smoke but not evidence

A selector can pass smoke and still fail evidence. Smoke is execution validation, not scientific validation.

11.5 RC2H failure: conservative anchor helps but context-only overrides remain unsafe

Anchoring is necessary, but the safe anchor set should include both context-gap and true global guard. Context-only global candidates should be restricted.

Story for a 12-year-old

Every failed version taught us which child fell down: class 2, class 4, class 5, or the selector itself. The failures were not useless. They were like footprints showing where the bridge is weak.

11.6 RC2JK failure: Pareto fallback over-protects clean MNIST

RC2JK made the guarded context-gap fallback mechanically explicit, but MNIST robust evidence showed that it could over-trigger ambiguity and select context-gap where a simple proto/no-repair policy was better. This is the clean-regime regression problem.

11.7 RC2JKb failure: clean escape remains too permissive on FashionMNIST

RC2JKb fixed the over-locking problem by separating context uncertainty from performance-risk ambiguity. It passed its audit gates and produced explicit traces, but FashionMNIST robust evidence showed that seeds 0 and 2 could still escape to true-guard when context-gap guarded was the better deployable candidate. This is the ambiguous-regime under-protection problem.

11.8 Meta-failure: selector regret, not candidate-capacity collapse

The repeated RC2J pattern is not random non-convergence. It is a stable diagnosis. The candidate pool contains strong policies for each regime, but the universal selector has not yet learned when to trust each family. This is why RC2L is the next scientific branch.

12 Current best scientific statement

A careful statement after RC2O-v1 is:

MMALS has demonstrated a robust-qualified Phase-1 regime-safe deployment controller on a four-dataset MNIST-family overlap-chain suite. The current best controller is RC2O-v1, which separates feature-only regime estimation from hard regime-conditional deployment. It releases proto/global no-repair behavior in clean or stable domain-shift regimes and locks the guarded context-gap family in ambiguous regimes. The result is not a proof of open-world continual learning and not a production certificate, but it is stronger than the previous Option A position: the selector is no longer merely an unresolved bottleneck inside the MNIST-family robust suite. It has passed the first controlled clean-versus-ambiguous regression qualification.

Table 18: Updated deployment statement after RC2O-v1.

Regime	RC2O-v1 deployable choice	Scientific interpretation
Clean MNIST	proto/global no-repair	avoid over-protective context-gap fallback; clean release recovered
Ambiguous FashionMNIST	guarded context-gap	block no-repair; protect min-task and class-2 weak boundary
Clean geometric shift RotatedMNIST	proto/global no-repair	deterministic rotation does not require guarded context-gap lock in robust run
Clean permutation shift PermutedMNIST	proto/global no-repair	complex repair mostly unnecessary; simple/global behavior is safe
Unknown or mixed future regime	guarded fallback or abstention	future conformal calibration should decide when the rule-based fallback is enough

Story for a 12-year-old

The mushroom forest now has a better weather rule. In clear weather, use the fast simple path. In fog, use the guarded path. If the weather is unknown, stop being brave and choose the safer path.

13 Next research roadmap

13.1 External-domain qualification

The immediate scientific constraint is that RC2O-v1 remains a MNIST-family robust qualification. The next evidence tier must test whether feature-only regime estimation remains meaningful on external datasets such as Split-CIFAR100, CORe50, TinyImageNet-style class-incremental splits, or small DomainNet-like domain shifts. The claim should not become “general continual learning” until the selector works outside the MNIST-family geometry.

13.2 Axis A: host-specialization audit

RC2O-v1 qualifies the deployment controller, not the causal specialization of the hosts. The next audit must test whether hosts are functionally specialized through route dominance, mutualistic gain, ablation impact, output divergence, representation separation, per-task single-host collapse controls, and true cross-seed checkpoint alignment. The patched Axis A notebook included in this package is intended for that purpose. It should not overclaim cross-seed specialization unless distinct per-seed checkpoint hashes are present.

13.3 Continual regime replay

The next long-form continual-learning proof should train and revisit multiple regimes sequentially: train regime 1, evaluate regime 1; train regime 2, evaluate regimes 2 and 1; train regimes 3 and 4, then evaluate 1, 2, 3, 4 and shuffled orders such as 2, 1, 4, 3. This would test not only accuracy and forgetting, but also route reuse, context posterior stability, host reuse, Hydro drift, LSA, Energy Probe behavior, memory cost, parameter cost, latency, and response-time cost.

13.4 RC2O-v2 / RC3: conformal regime-conditional calibrated selector

RC2O-v1 uses rule-based fallback and hard regime-conditioned deployment. The proposed RC2O-v2 or RC3 layer should convert regime confidence, selector regret, boundary reliability, Hydro/LSA stability, Energy Probe status, and memory-safety margins into conformal nonconformity scores. In engineer terms, RC2O-v2 would replace “my rules say this is safe” with “calibration on past cases says this decision is safe enough at the chosen error level.” In child-level terms, it gives the mushroom a danger meter learned from previous walks.

This conformal layer is not implemented in RC2O-v1. It is future work. Under standard exchangeability assumptions, it could provide finite-sample coverage for regime sets or safe-policy sets, allowing calibrated abstention when the current case does not resemble previously safe calibration cases.

13.5 Cost and production-readiness gap

The final engineering campaign must include accuracy, forgetting, selector regret, parameter count, model size, training time, inference latency, replay buffer cost, audit storage cost, and stability metrics. A useful MMALS system must justify its larger architecture by providing robustness, auditability, context inference, and safe fallback that simpler baselines do not provide.

Story for a 12-year-old

The next tournament is not just about who gets the most answers right. It asks who remembers old games, who knows when the forest is foggy, who explains the path, who costs too much food, and who can say: “I am not sure, so I will use the safe route.”

14 Response to the original four MMALS pillars

The initial motivation behind MMALS can be summarized by four pillars. The updated evidence supports each pillar differently.

Table 19: Status of the original MMALS pillars after Option A consolidation.

Original pillar	Current evidence	Current limitation
Catastrophic forgetting control	Fixed guarded candidates and replay-beating MMALS variants reduce forgetting in harder overlap-chain regimes.	Universal selector can still choose candidates with worse min-task and class-2 behavior.
Inferred-context routing	v1.0-RC1b and later candidates infer context from input/memory rather than relying on explicit task IDs at inference.	Context uncertainty is not identical to performance-risk ambiguity; selector calibration is needed.
Modular/cost-aware adaptation	Hosts, routes, transformations, readouts, and candidate policies expose where adaptation occurs; cost is measured rather than hidden.	MMALS remains larger and slower than MLP baselines; cost must be justified by harder regimes and auditability.
Auditability and explainability	Hydro, LSA, EER, Energy Context Probe, Drive persistence, manifests, and traces provide strong observability.	Audit signals are not yet calibrated enough to produce a universally reliable selector.

A fifth pillar has emerged from the evidence: **selector reliability**. Once candidate policies become strong and diverse, the scientific bottleneck becomes the reliability of the mechanism that chooses among them.

Story for a 12-year-old

The mushroom forest made progress on remembering, guessing the situation, using different trees, and explaining what happened. But a new problem appeared: choosing the best rule is itself a skill. The mushroom now needs to learn when to trust its own selector.

15 Limitations

This research program has important limitations:

1. Most claims are still MNIST-family claims.
2. Smoke runs are not evidence.
3. Evidence runs are not robust qualification.
4. The selector remains the main bottleneck; RC2JKb is an audit finding, not the frozen deployable selector.
5. The model is more expensive than MLP baselines.
6. Biological inspiration is an analogy, not a biological validation.
7. Current robust evidence is MNIST-family evidence; it does not yet establish open-world continual learning.
8. No claim of production readiness, safety certification, open-world learning, or industry level qualification is justified yet.

Story for a 12-year-old

The mushroom forest has not traveled the whole world yet. It has mostly practiced on small maps. Before we trust it in a real jungle, it needs harder maps, more races, and stricter judges.

16 Conclusion

MMALS began as a mycelium-inspired routing architecture and matured into an auditable continual-learning research program. The strongest conceptual contribution remains the distinction between route memory and functional memory. The strongest engineering contribution is the evidence harness: no-leakage policy selection, replay comparisons, weak-class diagnostics, context posterior auditing, Hydro stability traces, LSA Panel E, EER-style threshold diagnostics, Energy Context Probe, Drive persistence, manifests, and reproducible packages.

The June 2026 update changes the status of the story. Earlier Option A correctly concluded that the candidate pool was strong but the universal selector was not yet reliable. RC2O-v1 answers that specific clean-versus-ambiguous selector conflict inside the MNIST-family robust suite. It does not find one universal policy. Instead, it implements the more mature system view: infer the regime from validation/audit features, then deploy the policy family that is safe for that regime. This is why the distinction between backbone learning and deployment-policy selection matters. A trained

backbone can contain several usable behaviors; the deployed predictor is the trained backbone plus the selected policy.

The current best result is therefore:

RC2O-v1 is a robust-qualified Phase-1 regime-safe deployment controller across MNIST, FashionMNIST, RotatedMNIST, and PermutedMNIST overlap-chain robust runs.

The claim is still bounded. It is not production readiness, not an open-world proof, not host-specialization proof, and not a conformal guarantee. The next steps are clear: external-domain qualification, Axis A host-specialization audit, continual regime replay, and a future conformal/regime-conditional calibrated selector.

Story for a 12-year-old

The mushroom forest has learned a better rule: first understand the weather, then choose the path. It now knows not to use the guarded path everywhere and not to take shortcuts in the fog. But it still has to prove that its trees are truly specialized, that it can travel beyond small maps, and that it can measure when a new forest is too strange.

17 Milestone table with child-level story

Table 20: MMALS research progression.

Step	Scientific role	Story for a 12-year-old
v0.1	Structural routing toy model.	Build the first mushroom path between smart trees.
v0.2	Fair baselines and capacity checks.	Make sure the mushroom is not winning just because it has a bigger engine.
v0.3	Real fungal medium with hosts and transformations.	The mushroom messenger starts mixing advice from different trees.
v0.4	Warm-up and delayed mutualism.	Let each tree learn its job before forcing teamwork.
v0.5	Route memory and output distillation.	Remember both the road and the answer.
v0.6	Route-memory ablation on harder data.	Check whether remembering the road really helps.
v0.7	Functional route memory.	Check whether the package delivered by the road stayed the same.
v0.8	Evidence harness.	Organize a tournament and record the scores.
v0.9	Route-function disentanglement.	The same road can carry a different package.
v0.10	Cost and scalability.	A big forest must justify its cost.
v0.11	Comparative baselines.	Race the mushroom against known teams.
v0.12	Hopfield/frozen controls.	Test whether turning the forest to stone helps.

Step	Scientific role	Story for a 12-year-old
v0.13	Geometry and co-evolution diagnostics.	Use a microscope to see how paths stretch.
v0.14	Inferred context.	Guess the classroom without being told the subject.
v1.0-RC1b	Proto-first inferred-context candidate with guarded readout.	Choose paths by comparing today with old memories.
RC2b	Strict no-leakage policy selection.	Pick a strategy before looking at the final exam.
RC2c-RC2g	Risk-aware and boundary-safe selector repairs.	Stop one child from falling without pushing another down.
RC2H	RC1b-anchored conservative selector.	Keep the trusted plan unless a new plan clearly wins.
RC2I	Dual-anchor conservative selector.	Trust two safe teachers before accepting a new shortcut.
RC2J-RC2JF	Regime-aware selector, hard clarity, true-guard safe veto.	Learn when the forest is foggy and when it is clear.
RC2JG	Generic weak-boundary latent calibration candidate.	Build a small bridge only when practice memories show mixed roots.
RC2JH	Energy Context Probe and scale audit.	Use a compass to see whether the context valley is reachable.
RC2JK/RC2JKB	Bareto-safe fallback, clean escape, strict ambiguity floor.	Separate fog from real danger, but discover the voting rule is still imperfect.
RC2L	Selector-oracle gap and validation-feature reliability audit.	Check whether the mushroom’s practice notes can predict the best path.
RC2M-alpha	Boundary-first calibrated selector exploration and FashionMNIST robust package.	Give the mushroom a spreadsheet of practice clues.
RC2N	Clean-release and ambiguity-lock threshold ceiling.	Discover that one weather rule is not enough.
RC2O-v1	Feature-only regime estimator plus hard regime-conditional deployment.	First understand the weather, then choose the path.
Axis A next	Host-specialization audit using ablation, route swap, collapse controls, and cross-seed alignment.	Check whether different trees really have different jobs.
RC2O-v2/RC3 next	Conformal regime-conditional calibrated safe selector.	Give the mushroom a calibrated danger meter.

18 Reproducibility package contents

The accompanying GitHub-ready package contains:

- `paper/main.tex`: LaTeX source for this updated article;
- `paper/MMALS_Full_Story_PhD_Article_UPDATED_RC20_v1_Full_Chronicle.pdf`: compiled PDF;

- `paper/data/rc2o_v1_robust_four_dataset_summary.csv`: compact four-dataset robust summary;
- `metrics/`: extracted compact RC2O-v1 scorecards, regime-decision traces, EER summaries, Energy Probe summaries, and legacy RC2L/RC2M-alpha metrics;
- `notebooks/`: RC2O-v1 FashionMNIST robust notebook and patched Axis A host-specialization audit notebook;
- `docs/`: explanatory notes, RC2O-v1 full-story update, and preserved evidence PDFs;
- `legacy_inputs/`: the user-provided RC2M-alpha FashionMNIST robust package preserved for traceability;
- `previous_versions/`: previous PDFs/packages kept so the narrative is not lost;
- `README.md`: package map and update notes.

19 Notation

Symbol	Meaning
x	input
c	context, explicit in v0.x and inferred in v1 direction
$g_i(x)$	host representation
T_i	fungus transformation for host i
$r_i(x, c)$	route weight for host i
$z_f(x, c)$	mediated latent state
D_r	route drift
D_z	functional latent drift
D_y	output drift
$q(c x)$	inferred context posterior
$E(z, m, c)$	audit-only Energy Context Probe objective
LSA	Learning-Signal Alignment between expected and observed update allocation
EER	one-vs-rest threshold diagnostic used for weak-boundary audit
R_{sel}	selector regret relative to the candidate-pool oracle
$\Gamma(x)$	future conformal set of plausible regimes or safe policies
π_k	deployment policy selected from candidate pool Π
$h_\theta(x)$	trained backbone mechanism before deployment-policy selection

References

- Bebber, D. P., Hynes, J., Darrah, P. R., Boddy, L., and Fricker, M. D. (2007). Biological solutions to transport network design. *Proceedings of the Royal Society B*, 274, 2307–2315.
- Bronstein, M. M., Bruna, J., LeCun, Y., Szlam, A., and Vandergheynst, P. (2017). Geometric deep learning: going beyond Euclidean data. *IEEE Signal Processing Magazine*, 34(4), 18–42.
- Chaudhry, A., Ranzato, M., Rohrbach, M., and Elhoseiny, M. (2019). Efficient lifelong learning with A-GEM. *ICLR*.

- Hopfield, J. J. (1982). Neural networks and physical systems with emergent collective computational abilities. *PNAS*, 79(8), 2554–2558.
- Kirkpatrick, J. et al. (2017). Overcoming catastrophic forgetting in neural networks. *PNAS*.
- Li, Z. and Hoiem, D. (2017). Learning without forgetting. *IEEE TPAMI*.
- Lopez-Paz, D. and Ranzato, M. (2017). Gradient episodic memory for continual learning. *NeurIPS*.
- Rebuffi, S.-A., Kolesnikov, A., Sperl, G., and Lampert, C. H. (2017). iCaRL: Incremental classifier and representation learning. *CVPR*.
- Rusu, A. A. et al. (2016). Progressive Neural Networks. *arXiv:1606.04671*.
- Simard, S. W. et al. (1997). Net transfer of carbon between ectomycorrhizal tree species in the field. *Nature*, 388, 579–582.
- van de Ven, G. M., Tuytelaars, T., and Tolias, A. S. (2022). Three types of incremental learning. *Nature Machine Intelligence*.
- Harbonnier, G. (2026a). *MMALS Prototype 0.9: Route-Function Disentanglement and Functional Memory Validation*. Internal research notebook and evidence package.
- Harbonnier, G. (2026b). *MMALS Prototype 0.11: Comparative Evidence Harness*. Internal research notebook and evidence package.
- Harbonnier, G. (2026c). *MMALS v1.1-RC2b: Strict No-Leakage Policy-Selection Harness*. Internal research notebook and evidence package.
- Harbonnier, G. (2026d). *MMALS v1.1-RC2h: RC1b-Anchored Conservative Selector Harness*. Internal research notebook and evidence package.
- Harbonnier, G. (2026e). *MMALS v1.1-RC2J/RC2JE/RC2JF: Regime-Aware Selector, Hard-Clarity Veto, and True-Guard Safe Veto*. Internal research notebooks and evidence packages.
- Harbonnier, G. (2026f). *MMALS v1.1-RC2JG/RC2JH: Generic Weak-Boundary Latent Calibration and Energy Context Probe*. Internal research notebooks and evidence packages.
- Harbonnier, G. (2026g). *MMALS v1.1-RC2JKb: Clean-Regime Escape, Strict Ambiguity Floor, and Selector Audit Patch*. Internal research notebook and evidence package.
- Harbonnier, G. (2026h). *MMALS v1.1-RC2L: Selector-Oracle Gap and Validation-Feature Reliability Audit*. Internal research notebook and evidence package.
- Harbonnier, G. (2026i). *MMALS v1.1-RC2M-alpha: Boundary-First Calibrated Meta-Selector and FashionMNIST Robust Evidence Package*. Internal research notebook and evidence package.
- Harbonnier, G. (2026j). *MMALS v1.1-RC2O-v1: Feature-Only Regime Estimator and Hard Regime-Conditional Selector*. Internal research notebooks and robust evidence packages.